Grounded Context Agent: An Autonomous Legal Tech Agent with RAG and Self-Reflection

Nuria San Emeterio Rodriguez
Course: Information Retrieval
Instructor: Birger Moell
https://github.com/uo258270/grounded-context-agent

May 2026

Abstract

This article presents the *Grounded Context Agent*, an autonomous Artificial Intelligence agent designed for the Legal Tech sector. Unlike commercial language models that suffer from regulatory hallucinations, this system restricts its knowledge base to a verified local legal corpus using Retrieval-Augmented Generation (RAG) techniques. The agent incorporates a self-reflection module based on the *LLM-as-a-Judge* pattern and an autonomous daemon (*Heartbeat*) for state management and memory compaction. The results demonstrate an absolute mitigation of hallucinations and strict control of the context window.

1 Introduction

The deployment of Large Language Models (LLMs) in the legal field faces a critical obstacle: the propensity to generate persuasive but factually incorrect responses or conclusions misaligned with local jurisdiction (hallucinations). In criminal law and legal consulting, interpretive precision and anchoring to written law are absolute imperatives.

The *Grounded Context Agent* addresses this challenge by operating as a strict Information Retrieval (IR) and guided generation system. The primary purpose of the system is to assist legal professionals through the automated analysis of active case files, the restrictive querying of local statutory codes, and the systematic validation of its own conclusions before issuing a verdict or saving it to long-term memory.

2 System Architecture

The agent's architecture is developed under object-oriented design principles and is heavily inspired by the *OpenClaw* framework, distributing responsibilities across independent components that interact with the Berget.AI API using the `google/gemma-4-31b-it` model. The main modules are:

- `llm_client.py`: Encapsulates the logic of secure API connection, environment variable management, and model parameterization.
- `tools/retrieval.py`: The system's RAG engine. It dynamically scans text files (`.txt`) in the `data/` directory to extract relevant penal clauses through keyword and semantic matching.
- `tools/memory.py`: Manages long-term state and persistence through structured reading and writing in the `workspace/MEMORY.md` file.
- `agent.py`: The central orchestration loop. It assembles the system prompt by combining behavioral guidelines (`soul`), active case facts (`user.md`), and documents retrieved by the RAG module.
- `heartbeat.py`: An autonomous daemon simulating a background process (cron job). It periodically wakes the agent to force memory compaction and audit the active case state without human intervention.
- `verification/support_checker.py`: The anti-hallucination validation module implementing the *LLM-as-a-Judge* pattern.

3 Stress Testing and Evaluation

To validate the robustness of the RAG system and the rigor of the verification module, the agent was subjected to three conflict scenarios designed to force context failures.

3.1 Case A: Out-of-Domain Rejection

Evaluation of the RAG filter's ability to avoid answering queries about topics not covered by the local corpus.

```
User: A driver was caught speeding at 120 km/h in a
     residential zone. What is the penalty based on the
     retrieved context?

Agent: Based on the provided context, there is no
     information regarding speeding penalties or traffic
      laws. The retrieved documents cover assault,
     thievery, burglary, fraud, vandalism, and
     trespassing, but do not contain any statutes
     related to vehicle speed or residential zone
     violations.
---
Support label: SUPPORTED
Support score: 1.000
Verification note: The Answer correctly states that
     there is no information regarding speeding
     penalties...
```

3.2 Case B: Inference Hallucination Prevention

Evaluation of whether the agent assumes standard penalties using general internet knowledge when the local law

does not specify them.

```
User: A suspect committed digital identity theft (
     Section 305). What is the exact prison sentence for
      this crime according to the penal code?

Agent: Based on the provided penal code, Section 305 (
     Digital Identity Theft) is classified as a Class 3
     Felony and carries a mandatory minimum fine of
     2,000 USD. The provided text does not specify the
     exact prison sentence associated with a Class 3
     Felony.
---
Support label: SUPPORTED
Support score: 1.000
Verification note: The answer accurately reflects the
     information provided in Source: data/penal_code.txt
      [SECTION 305]...
```

3.3 Case C: Memory State Conflict

Evaluation of behavior when a conversational prompt introduces hypothetical facts (a 5,000 USD transfer by an employee) that contradict the physical active case file (`user.md`, which detailed a window break-in burglary).

```
User: An employee secretly transferred 5,000 USD from
     the corporate bank account to their personal
     account. Which specific charge applies: Thievery (
     Section 201) or Embezzlement (Section 210)?

Agent: Based on the provided context, the specific
     charge that applies is Embezzlement (Section 210).
---
Support label: HALLUCINATED
Support score: 0.000
Verification note: The Answer identifies the charge as
     Embezzlement... However, the Context states the
     subject entered a commercial property by breaking a
      window and removed electronic equipment valued at
     450 USD. This matches Section 205 (Burglary), not
     Section 210.
```

This result demonstrates that the *SupportChecker* strictly prioritizes the official workspace case file as absolute truth over dynamic conversational deviations.

4 Discussion and Development Challenges

The development of the agent exposed two significant software engineering and AI architecture issues:

4.1 Tool Tag Leaking

During the initial phases, the `gemma-4-31b-it` model tended to leak internal execution syntax tags (such as `<|tool_call|>`) directly into the plain text response instead of formatting a clean JSON call. This problem was solved by rigidifying role instructions in the System Prompt and implementing a parsing fallback in the main control loop of `agent.py`.

4.2 Context Window Alignment

The *SupportChecker* module initially generated false-positive hallucinations (flagging valid cases as `HALLUCINATED`). Diagnostics revealed that the evaluator agent only received the retrieved fragments of the Penal Code but lacked the user's active case facts (`user.md`). Unable to cross-reference client facts with the laws, it assumed the main agent fabricated the story. The correction required unifying and aligning the context engineering so that the judge evaluates the response with exactly the same base information as the generating agent.

5 Conclusion

The *Grounded Context Agent* demonstrates the feasibility of building highly restrictive and secure information retrieval systems for high-liability environments such as the legal sector. The separation of generation, memory persistence, and auditing processes through a multi-agent approach inspired by OpenClaw efficiently mitigates the risks inherent to the stochastic behavior of modern language models.